

Pytania z przedmiotu Oprogramowanie Systemowe

1. Omów mechanizm journalingu. Wymień nazwy systemów plików, w których jest stosowany.

Przy użyciu księgowania dane nie są od razu zapisywane na dysk, tylko zapisywane w swoistym dzienniku/kronice (ang. journal). Dzięki takiemu mechanizmowi działania zmniejsza się prawdopodobieństwo utraty danych: jeśli utrata zasilania nastąpiła w trakcie zapisu - zapis zostanie dokończony po przywróceniu zasilania, jeśli przed - stracimy tylko ostatnio naniesione poprawki, a oryginalny plik pozostanie nietknięty. De facto księgowanie jest formą realizacji zapisu transakcyjnego - jaki zwykle oferowały transakcyjne bazy danych (DB2, IMS, Oracle, RDB i inne), czy serwery transakcyjne (TTS, CICS).

Systemami plików z księgowaniem są: ext3, ext4, NTFS, HFS+ (system plików Apple dla Mac OS).

2. Omów proces uruchamiania komputera z BIOSem (od wciśnięcia przycisku zasilania do załadowania systemu operacyjnego).

- BIOS POST – pierwsza wykonana czynność. Zazwyczaj testuje integralność sprzętu (rejstry, pamięć, klawiatura etc.) i wyłącza niemaskowalne przerwanie.
- Załadowanie MBR – Master Boot Record – pierwszy sektor dysku twardego zawierający program rozruchowy
- Załadowanie VBR – Volume Boot Record – jak wyżej, tyle, że na partycjach
- odpalenie boot menagera – umożliwia załadowanie OS (wyświetla też menu wyboru)
- Przetworzenie BCD – Boot Configuration Data – dane do użycia w trakcie bootowania
- winload.exe – bootloader windowsa przejmuje pałeczkę, on ładuje wszystko aż do sterowników boot-class
- ładowanie SYSTEM hive – zawiera ustawienia nie przyporządkowane do żadnego użytkownika
- samosprawdzenie integralności winload.exe
- załadowanie i sprawdzenie ntoskrnl.exe,hal.dll – jądro systemu i Hardware Abstraction Layer
- ładowanie, weryfikacja i inicjalizacja importów ntoskrnl.exe
- skanowanie rejestrów
- ładowanie, weryfikacja i inicjalizacja sterowników boot-class
- ntoskrnl.exe jest inicjalizowany – w końcu. Przejmuje pałeczkę.
- inicjalizacja podsystemów, zbudowanie struktury systemu
- skanowanie rejestrów
- ładowanie, weryfikacja i inicjalizacja sterowników i usług „system class”
- smss.exe – Session Manager Subsystem – tworzy zmienne środowiskowe i generalnie przejmuje pałeczkę od tego momentu
- odpalenie aplikacji startupowych określonych przez wpis BootExecute w rejestrze
- załadowanie jądra podsystemu Win32K.sys
- załadowanie znanych DLL'ek
- podział:
- Session 0 smss.exe odpala
 - csrss.exe – client-server run-time subsystem – odpowiada za wątki m.in
 - wininit.exe – program inicjalizujący windowsa
 - lsass.exe, lsm.exe, services.exe -jakieś tam rzeczy dla windowsa
- Session 1 smss.exe odpala
 - csrss.exe - client-server run-time subsystem – odpowiada za wątki m.in
 - winlogon.exe - wiadomo
 - logonUI.exe – jeszcze bardziej wiadomo
 - userinit.exe – program inicjalizujący usera

3. Przedstaw charakterystykę systemów opartych o EFI.

EFI: Extensible Firmware Interface - definiuje interfejs pomiędzy systemem operacyjnym a firmwarem sprzętu, następca BIOS.

Systemy EFI:

- są niezależne od implementacji sprzętu
- nie wymagają trybu rzeczywistego procesora do rozruchu
- wykorzystują płaski model pamięci, gdzie cała dostępna pamięć może być zaadresowana
- nie nakładają limitów na całkowity rozmiar pamięci ROM. Sterowniki i aplikacje mogą być ładowane z dowolnego źródła w dowolne miejsce przestrzeni adresowej.
- Zawierają dodatkową powłokę (EFI shell)
- Ewoluuja od VGA do UGA (Universal Graphics Adapter)
- Jako pierwsza załadowana zostaje cienka warstwa PEI, która wykonuje większość rzeczy związanych z rozruchem komputera (inicjalizacja chipsetu, pamięci, enumeracja szyn danych etc.)
- Następnie EFI przygotowuje DXE (Driver Execution Environment) by zapewnić podstawowe funkcje, których mogą

użyć sterowniki EFI

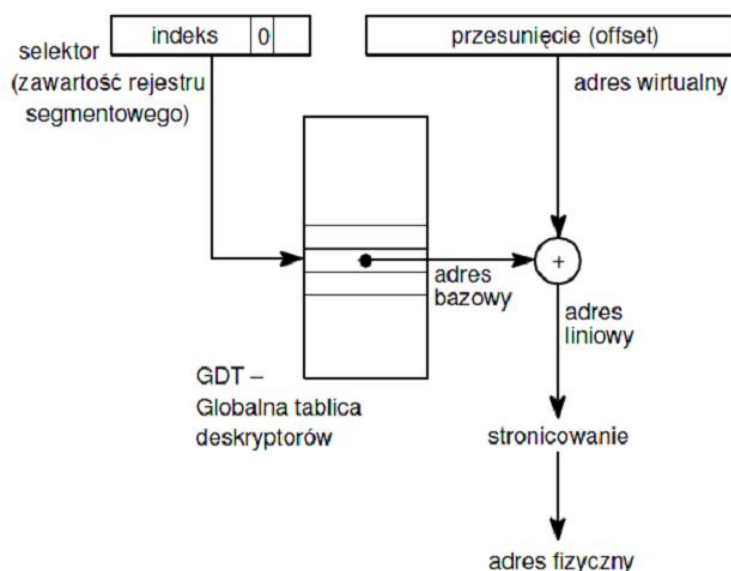
- Wykrywane są partycje
- Następuje uruchomienie bootloadera i jak wszystko ok, to przekazujemy sterowanie do OSa. W przeciwnym wypadku uruchamiany jest EFI Shell.
- Umożliwiają bootowanie z dużych dysków (>2TB)
- Posiada własny boot loader
- Szybszy proces bootowania
- Wsparcie dla sieci, bardziej przyjazny interfejs graficzny.

4. Wyjaśnij różnicę między liniowym a fizycznym adresem w pamięci operacyjnej.

Adres fizyczny jest adresem bezpośrednio określającym położenie segmentu w pamięci, przez ten adres maszyna musi się do odpowiedniego segmentu odwoływać.

Adres liniowy natomiast składa się z numeru katalogu(10-bit), odnoszącego się do Directory Entry w Page Directory, numeru tabeli(10-bit), odnoszącego się do Table Entry w Page Table i offsetu(12-bit). Taki adres, zapisany w Dekryptorze Segmentu w Global Descriptor Table może zostać wykorzystany przez sprzęt dopiero po przetłumaczeniu z użyciem wyżej wymienionych stron na adres fizyczny.

W przypadku wyłączenia stronicowania, adres liniowy jest tożsamy adresowi fizycznemu wyznaczonemu w opisany sposób.



5. Wymień metody ukrywania danych (plików) w systemach plików i opisz pokrótce ideę każdej z nich:

Out of band: zapis w niewykorzystanym i niewidocznym przez system miejsce. Wynika z rozmiaru klastra - pojemność dysku jest skokowa i przy partycjonowaniu może nam zostać trochę nieużywanego miejsca.

In-band: zapis w miejscu zarezerwowanym dla pliku - np klaster 4kb, plik 1kb i zostaje wolne 3kb (plik nawet jednobitowy zajmuje na dysku cały klaster)

Ukrywanie partycji: (np w laptopach partycja do odzyskiwania systemu) typ partycji ustawiamy na jakiś swój i wtedy system nie wie co to jest i zlewa.

Ukrywanie w warstwie aplikacyjnej: pewnie ukrywamy jakieś dane w obszarze, gdzie niby aplikacja ma być.

6. Podaj cele stosowania struktury GDT.

- uzyskanie liniowej przestrzeni pamięci, niezależnie od fizycznego rozproszenia danych
- umożliwienie obsługi większych ilości pamięci niż przy adresowaniu fizycznym
- zwiększenie funkcjonalności i kontroli nad tym, co się dzieje - dodanie opisów segmentów - kod/dane, opisy przywilejów, czy segment jest w pamięci
- zwiększenie bezpieczeństwa

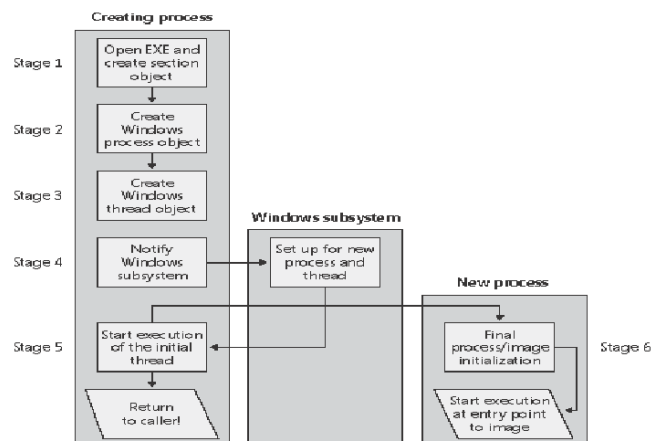
7. Opisz rolę bloku EPROCESS (blok wykonawczy procesu).

Process Environment Block zawiera struktury danych (sam w sumie też nią jest), które mają zastosowanie dla obsługi całych procesów, takie jak kontekst globalny, parametry startowe, struktury danych dla loadera obrazu programu, adres

bazowy obrazu programu, obiekty synchronizujące, informacje o tym, czy program jest aktualnie debugowany, oraz wiele innych. Duża część tej struktury danych nie jest udokumentowana przez Microsoft.

8. Opisz przepływ sterowania podczas tworzenia procesu w systemie Windows XP.

- Otwarcie pliku .exe i utworzenie obiektów sekcji
- Utworzenie obiektów procesu
 - Ustawienie struktury EPROCESS
 - Utworzenie inicjalnej przestrzeni adresowej
 - Utworzenie bloku kernela procesu
 - Ukończenie tworzenia przestrzeni adresowej
 - Ustawienie PEB
 - Ukończenie konfiguracji EPROCESS
- Utworzenie obiektu wątku
- Powiadomienie podsystemu o nowym procesie
- Utworzenie wątku głównego
- Utworzenie kontekstu i uruchomienie wątku



9. Jaka rolę w systemie plików NTFS pełni nadrzędna tablica plików (MFT)?

Master File Table (MFT) - metaplik zawiera wpisy (rekordy plikowe) opisujące każdy plik/katalog znajdujący się na partycji NTFS w postaci rekordu plikowego (1024 -4096 bajtów). Rekord plikowy składa się z nagłówka oraz sekwencji występujących po sobie atrybutów w postaci pary (atrybut, wartość).

Charakterystycznym elementem nagłówka rekordu plikowego są znaki „FILE” (w kodzie ASCII) umieszczone na początku nagłówka.

Nagłówek zawiera kilkanaście pól o różnym przeznaczeniu, m.in. podany jest faktyczny rozmiar rekordu plikowego (4-bajtowe pole o offsecie 18H) oraz zaalokowany rozmiar rekordu plikowego (4-bajtowe pole o offsecie 1CH).

Plik 0	MFT
Plik 1	Fragment MFT (kopia)
Plik 2	Pliki systemowe (NTFS metadata files)
Plik 16	Pliki i katalogi użytkownika

10. Budowa i rola rekordu plikowego w NTFS.

Atrybuty w rekordzie plikowym:

- Informacje standardowe – znaczniki czasu, ilość łączy, prawa dostępu
- Nazwa pliku/katalogu
- Informacje o woluminie – nazwa woluminu
- Dane – dane pliku
- Indeks główny – używany do implementowania katalogów i indeksów
- Mapa bitowa – dane tego atrybutu zawierają pole bitowe

Rodzaje atrybutów:

- rezydentne - ich rozmiar nie przekracza rozmiaru rekordu plikowego

-nierezydentne - większe od rozmiaru rekordu plikowego, wówczas w miejscu danych znajduje się lista wskaźników do kolejnych części danych.

Rekord plikowy

nagłówek
atrybut
atrybut
atrybut
znacznik końca rekordu plikowego:FFFFFFFFH

11. W jaki sposób tablica alokacji plików (FAT) opisuje położenie plików na dysku?

File allocation table (FAT) zawiera informacje o stanie wszystkich klasterów obszaru danych dysku. Każda pozycja tablicy używana przez plik wskazuje następną pozycję w FAT dla tego pliku (metoda listowa). Sekwencja numerów kolejnych pozycji jest taka sama jak numery wiązek składających się na cały plik. Pierwsze dwie pozycje w tablicy są zarezerwowane. Trzecia zawiera informację o pierwszym klastrze obszaru danych (numer 2). Każdy wpis ma 32 bajty i zawiera m.in:

- krótką nazwę pliku
- czas i datę modyfikacji/utworzenia/dostępu
- atrybuty pliku
- rozmiar pliku

12. Omówić strukturę partycji rozszerzonej na dysku.

Pojedyncza partycja może pomieścić informacje tylko o 4 dyskach logicznych. Celem zwiększenia tej liczby zdefiniowano partycję rozszerzoną w skład, której może wchodzić wiele dysków logicznych, które przez użytkownika są widziane, jako zwykłe dyski. Partycje rozszerzone są stosowane w systemie Windows.

Na początku partycji rozszerzonej znajduje się sektor o specyficznej strukturze, w którym zawarta jest tablica partycji, położona w tym samym miejscu, co w przypadku MBR'a. W tablicy tej wykorzystywane są tylko dwie pierwsze pozycje.

Pierwszy element tablicy partycji wskazuje adres początkowy dysku logicznego.
 Drugi – sektor dyskowy o takiej samej strukturze, stanowiący kolejny element listy dysków logicznych.

26 619 768	Dysk logiczny F: (2 GB)			
	0	0	0	4
	0	0	0	3
	0	0	0	2
26 619 705	07H	63	4 096 512	1
18 426 618	Dysk logiczny E: (4 GB)			
	0	0	0	4
	0	0	0	3
	05H	10 233 405	4 096 575	2
18 426 555	07H	63	8 193 087	1
16 386 363	Dysk logiczny D: (1 GB)			
	0	0	0	4
	0	0	0	3
	05H	2 040 255	8 193 150	2
16 386 300	0BH	63	2 040 192	1

x

Element zawierający 0 w polu identyfikatora systemu operacyjnego druga kolumna wskazuje koniec tej listy.

13. Opisz mechanizm obsługi przerwania na poziomie jądra w systemie Windows XP. Wyjaśnij rolę zmiennej IRQL.

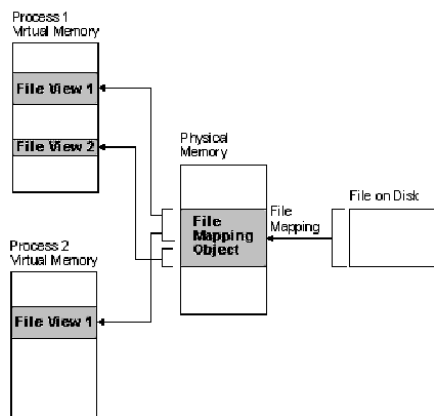
Przerwania w systemie Windows są relizowane poprzez wirtualizację obsługi przerwania (przerwania od sprzętu są dostarczane w odpowiedniej formie z warstwy HAL). Przerwania mogą mieć różny priorytet (określa to zmienna IRQL - Interrupt Request Level), przerwanie o niższym priorytecie mogą zostać przerwane (zamaskowane) poprzez przerwanie o wyższej wartości IRQL.

Kiedy wystąpi przerwanie, jest ono obsługiwane przez funkcję ISR (Interrupt Service Routine). Struktura IDT (Interrupt Dispatch Table - związana z konkretnym procesorem) kontroluje która funkcja obsłuży przerwanie z danym IRQL. Sterowniki łączą ISR z konkretnym IRQL poprzez stworzenie obiektu przerwania (interrupt object) i przekazując go do kernela. Łączy go on z odpowiednim wpisem w tablicy IDT (z każdym IRQL może być związane więcej niż 1 przerwanie), w przypadku odpięcia sterownika(urządzenia) obiekt przerwania jest po prostu usuwany z tablicy.

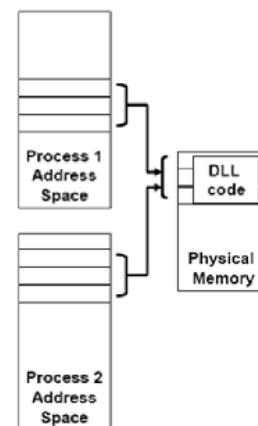
14. Przedstaw działanie i zastosowanie odwzorowywania plików (file mapping). Podaj przykładowe wykorzystanie tego mechanizmu w praktyce.

Mapowanie pliku przypisuje zawartość pliku do części wirtualnej przestrzeni adresowej procesu. Mapowanie pozwala procesowi korzystać z dostępu swobodnego oraz sekwencyjnego do pliku. Tworzenie widoku (fragment wirtualnej przestrzeni adresowej) pozwala na efektywną pracę z dużymi plikami, np. baz danych – nie ma potrzeby mapowania całego pliku, a tylko jego fragmentu. Wątki mogą używać pamięci zmapowanego pliku do współdzielenia danych. Używanie mapowania poprawia wydajność, bo korzystamy z widoku, który znajduje się w pamięci, a nie bezpośrednio z pliku na dysku. Ponadto nawet jeśli rozmiar pliku jest większy od rozmiaru pamięci możemy swobodnie się do niego odwoływać.

Mapowanie kodu bibliotek



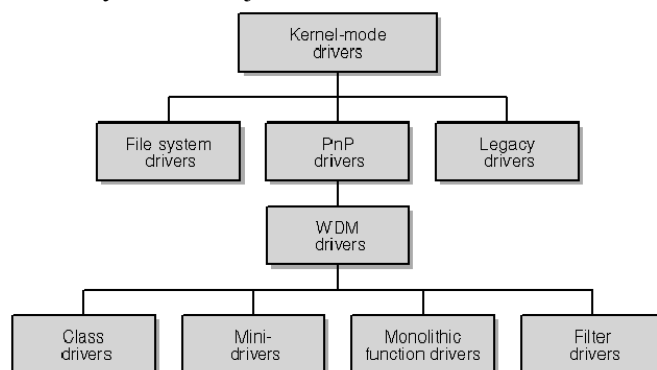
Relationship between the file on disk, a file mapping object, and a file view.



Zastosowanie: mapowanie kodu bibliotek DLL.

16. Przedstawić klasyfikację sterowników w systemie Windows XP i podać ich charakterystyki.

- **Sterowniki systemu plików:** implementują system plików dla dysku lokalnego lub podłączonego sieciowo.
- **Sterowniki typu Legacy:** Kontrolują sprzęt bez żadnego wsparcia od strony systemu operacyjnego (Sterowniki dla nieistniejącego sprzętu są dodane do systemu właśnie jako legacy drivers)
- **Sterowniki PnP:** Implementują protokół Plug and Play.
- **Sterowniki WDM (Windows Driver Model)** to sterowniki PnP, które dodatkowo implementują protokoły zarządzania energią i są zgodne zarówno z Windows 98/ME i Windows 2K/XP. To definicja - w praktyce sterowniki WDM nie zawsze oferują systemowi operacyjnemu wszystkie funkcje
- **Sterowniki klas:** Łączą wspólną funkcjonalność dobrze zdefiniowanych klas urządzeń (np. kart sieciowych - NDIS class drivers). Producenci sprzętu w większości nie piszą własnych sterowników klas, ale używają jednego z dostarczonych przez Microsoft.
- **Minidriver:** Pisane przez producentów jako wsparcie dla sterowników klas.
- **Sterowniki monolityczne (Monolithic function driver):** Jedne sterownik dla całego urządzenia (systemy typu embedded).
- **Sterowniki Filtrów (filter driver):** Dodają i/lub modyfikują zachowanie urządzenia



17. Sposoby rejestracji sterownika w systemie Windows.

- ZwSetSystemInformation() -Pozyskujemy adres funkcji z ntdll.dll za pomocą GetProcAddress. Nie można usunąć sterownika aż do restartu.
- Wykorzystując Service Control Manager - CreateService API Sc.exe create nazwa binpath=ścieżka parametry
- Zapis do \Device\PhysicalMemory

18. Omówić implementację pamięci wirtualnej za pomocą segmentacji.

Polega na przechowywaniu niektórych segmentów programu w pamięci dyskowej i sprowadzaniu ich do pamięci operacyjnej, gdy stają się potrzebne.

P					W	A
7	65	4	3	2	1	0

Deskryptor segmentu - bajt dostępu

Obecność segmentu w pamięci operacyjnej wskazuje bit P(present). Jeżeli wystąpi odwołanie do deskryptora z P = 0 to pojawia się wyjątek(przerwanie procesora), w takiej sytuacji system odszukuje wolny obszar pamięci (lub ją zwalnia), pobiera potrzebny segment do pamięci i powraca do realizowanego programu.

Aby racjonalnie zorganizować wymianę segmentów potrzeba mieć informację o statystykach wykorzystania poszczególnych segmentów, do zbierania informacji pomocny jest bit A(access). Ustawiany jest on gdy do rejestru segmentowego zostanie wpisany selektor wskazujący rozpatrywany deskryptor. System operacyjny (nie procesor) zeruje go w obserwowanych segmentach oraz okresowo odczytuje jego wartość i wykonuje zerowanie. Jeżeli odczytano kilkakrotnie 0, znaczy to iż segment jest nieużywany.

Istotny w implementacji jest również bit W, system może początkowo przypisać wszystkim segmentom atrybut „tylko do odczytu”(W=0), każda próba zapisu do danego segmentu wywoła wyjątek, może on sprawdzać „legalność” operacji i jeżeli zdecydujemy się na zapis ustawiamy bit W. Daje to informacje o tym iż zawartość segmentu została zmieniona – jeżeli istnieje potrzeba usunięcia go z pamięci należy go przepisać na dysk, w przeciwnym wypadku zawartość segmentu w pamięci i na dysku jest jednakowa i nie ma potrzeby kopiowania go.

19. Jakie zalety ma realizacja pamięci wirtualnej za pomocą stronicowania w porównaniu z realizacją za pomocą segmentacji?

Segmentacja:

- deskryptor zawiera adres bazowy i rozmiar.
- segmenty różnych rozmiarów i słabo to systemowi przerywać, pamięć się fragmentuje

Stronicowanie:

- zamiast adresu bazowego wpis w katalogu stron i indeks wpisu (deskryptor = adres fizyczny katalogu stron + indeks w tablicy stron + offset)
- odwołanie przez katalog stron -> tablica stron -> adres fizyczny (działa jak segregator z zakładkami segregator -> zakładka -> strona)
- każdy procesor ma swój katalog stron (segregator) w rejestrze CR3

- w deskrypcie bit P - czy segment jest w pamięci -> zwalnianie ostatnio nieużywanych stron i ładowanie potrzebnych z pamięci. Proces odwołując się do ramki wstawia tam 1, system operacyjny go zeruje i robi sobie jakieś statystyki.
- rejestr cieni - procesor zapisuje sobie adresy rzeczywiste ramek, żeby nie robić za każdy raz tej translacji.
- tablica ostatnich adresów - używana jeśli nowy adres różni się tylko offsetem
- segmenty składają się ze stałego rozmiaru, zazwyczaj 4kb. Łatwiej jest systemowi zarządzać wieloma małymi kawałkami niż jednym dużym.

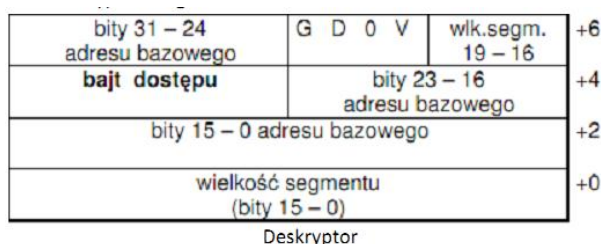
Segmentacja powoduje fragmentację pamięci (uboczny skutek wymiany segmentów o różnych rozmiarach), co nie ma miejsca przy stronicowaniu.

Poszczególne procesy mają dostęp do wyłącznie swoich własnych obszarów pamięci (odwołania przechodzą przez tablicę transformacji adresów i kierowane są pod obszar pamięci procesu).

20. W jaki sposób w architekturze IA-32 została zrealizowana koncepcja rejestru bazowego i granicznego jako metody izolacji procesów?

Rejestr bazowy przechowuje najmniejszy dopuszczalny adres fizyczny pamięci, a rejestr graniczny zawiera rozmiar obszaru pamięci (w innych architekturach może to być największy dopuszczalny adres fizyczny).

W IA-32 koncepcja realizowana jest poprzez określenie adresu bazowego i rozmiaru segmentu w odpowiednich polach deskryptora segmentu.

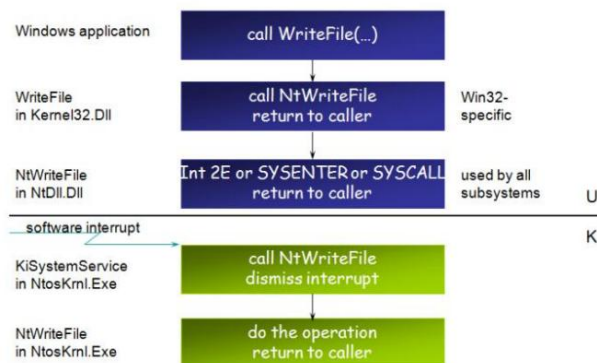


Uzyskujemy dzięki temu obszar do którego ma dostęp konkretny proces i poza który nie może się odwołać, w takim wypadku procesor zgłosi wyjątek.

Wartości w rejestrach ustawia SO podczas przełączania kontekstu.

22. Opisz mechanizm wywołania funkcji systemowej na poziomie jądra systemu operacyjnego Windows XP.

- Wywołanie funkcji Windowsowej np. WriteFile. Przez systemowe biblioteki dochodzi to do NTDll
- NTDll wykonuje funkcję Int2E, SYSENTER lub SYSCALL, która powoduje tzw. "przerwanie programowe"
- Systemowy dispatcher obsługuje przerwanie
 - Szuka odpowiedniego wpisu w tablicy przerw
 - Wpis zawiera numer tablicy i numer indeksu w tej tablicy - tam znajduje się API funkcji systemowej
 - Po takim odwołaniu funkcja zostaje uruchomiona



23. Przedstaw działanie procesu planisty w systemie Windows XP.

Istnieją 32 kolejki wątków oznaczające priorytety (31 najwyższy), 31-16 to real time, kolejne 15 zmiennych, i najniższy zarezerwowany dla wątku podstawowego. planista zawsze wybiera wątki o najwyższym priorytecie zgodnie z algorytmem 'round robin'. Po upływie kwantu czasu planista określa czy należy zmienić priorytet aktualnie wykonywanemu wątkowi, następnie rozpoczyna wykonywanie kolejnego wątku.

Każdy wątek ma dwa priorytety: bazowy i bieżący. Nowy wątek dziedziczy priorytet procesu (bazowy). Procesy zużywające dużo zasobów procesowa przenoszone są do kolejek o niższym priorytecie.

Zwiększenie wartości priorytetu:

- zakończenie operacji IO (+X w zależności od sterownika)
- wątek GUI dostaje focusa +2
- proces gotowy do działania nie działał od 4s (=15 i podwojenie kwantu)
- dostał event lub opuszczenie semafora +1

Podczas zmian wartości procesu jego priorytet nie może przekroczyć 15, co cykl wartości są obniżane o 1 aż do bazowego.

Taktyki szeregowania:

- dobrowolne przełączenie - gdy wątek wywoła coś jak wait()
- koniec kwantu - planista sprawdza czy nie wpierdolił za dużo i mu nie obniżyć

- do kolejki wpadł wątek o wyższym priorytecie -> przerywany jest akt wykonywany o niższym

24. Wymień i scharakteryzuj metody łączenia kodu bibliotecznego z kodem programu.

- Linkowanie statyczne polega na dołączeniu odpowiedniego kodu funkcji do programu na etapie kompilacji.
- Linkowanie dynamiczne polega na dołączaniu do programu jedynie odpowiednich odwołań do funkcji bibliotecznych. Sam kod funkcji nie jest dołączany do programu na etapie kompilacji. Przy uruchomieniu programu, biblioteka jest natomiast ładowana do pamięci i udostępnia odpowiednie funkcje pracującemu programowi.

Podstawową zaletą linkowania dynamicznego jest to, że jest wymagany jeden egzemplarz biblioteki w pamięci dla wszystkich programów które z niej korzystają. Wadą jest to, że program linkowany dynamicznie jest zależny od dostępności zewnętrznym plików (bibliotek) i nie będzie działał poprawnie jeżeli ich nie znajdzie.

Metoda dynamiczna może być w 2ch wariantach: RTL(run) i LTL(load) w czasie wykonania i w czasie ładowania.

RTL: LoadLib("dsd.dll");

LTL: przy ładowaniu programu system sam nam załaduje

25. Opisz budowę pliku PE.

Jest to wspólny format dla plików wykonywalnych i bibliotek dll w systemie Windows. Zawiera listę funkcji które daje i które chce. Plik składa się z:

- sygnatury "PE\0\0"
- małego programu dosowego wypisującego ze program wymaga systemu Windows.
- nagłówek COFF(typ procesora,liczba sekcji,rozmiar opcjonalnego nagłówka, bit exec czy dll)
- opcjonalny nagłówek
- katalogi danych (importy,exporty)
- nagłówki sekcji
- sekcje

26. Przedstaw kolejne kroki w komunikacji pomiędzy aplikacją użytkową a sprzętem w systemie Windows.

Aplikacja użytkownika wywołuje funkcję z Windows API, odwzorowywaną na funkcje z ntdll.dll. Następnie musi zostać wywołana funkcja na poziomie kernela (wyższy poziom uprzywilejowania). Odbywa się to za pomocą mechanizmu deskryptora bram. Żądanie trafia do warstwy HAL (hal.dll), która już ogarnia komunikacje ze sprzętem.

27. Opisz proces ładowania programów do pamięci operacyjnej.

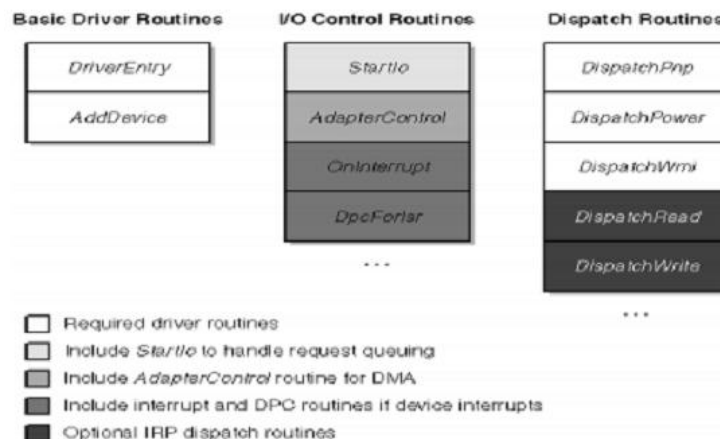
- ładowanie bezwzględne
 - cały kod programu musi być w pełni określony w chwili ładowania go do pamięci;
 - programista może posługiwać adresami fizycznymi lokacji pamięci, lub te adresy te mogą wyznaczone przez kompilator (assembler) i linker;
 - niezbędna jest jednak znajomość adresu początkowego obszaru pamięci, do którego zostanie wpisany program;
- ładowanie relokowalne
 - decyzję o położeniu programu w pamięci podejmuje się podczas ładowania
 - w takim przypadku program można umieścić w dowolnym obszarze pamięci;
- ładowanie dynamiczne
 - jest charakterystyczne dla współczesnych wielozadaniowych systemów operacyjnych, w których stosuje się pamięć wirtualną;
 - wówczas poszczególne fragmenty programu mogą być rozproszone w pamięci operacyjnej, a część z nich może być przechowywana na dysku;
 - ten sam fragment programu, jeśli zostanie ponownie przepisany z dysku do pamięci operacyjnej, może pracować w obszarze o innych adresach fizycznych;
 - realizacja ładowania dynamicznego wymaga więc skomplikowanych sprzętowych mechanizmów transformacji adresów (zob. stronicowanie).

28. Omów budowę podstawowego szkieletu sterownika urządzeń WDM w systemie Windows.

Procedury obowiązkowe:

- Driver Entry - funkcja inicjalizująca działanie sterownika. Jej zadanie to wskazanie SO, gdzie znajduje się niezbędne funkcje (np. AddDevice). W praktyce oznacza to wskazanie SO przez sterownik które funkcje może aplikacja użytkownika wykonać
- AddDevice - tworzy obiekt urządzenia (device object) oraz instancję rozszerzenia urządzenia (device extension)
 - rejestruje jeden lub więcej interfejsów urządzeń

- wpisuje device extension i flagi (flags member)
- dołącza device object do stosu urządzeń (device stack)



29. Ile stosów ma procedura w trybie chronionym? Wyjaśnij dlaczego tak się dzieje.

Dana procedura wykonywana jest przez wątek. Każdy wątek posiada dwa stosy – po jednym w trybie jądra i użytkownika. Powodem tego jest bezpieczeństwo – nic, co działa w trybie użytkownika nie ma dostępu do danych z trybu jądra.

30. Opisz proces ładowania sterowników dla urządzeń PnP.

- podczas startu system ładuje sterownik magistrali i przepytuje każde urządzenie na portach PnP, tworzy i ładuje jego device object.
- każde urządzenie ma swój unikalny łańcuch i zostawia ślad w rejestrze - adres drivera
- system szuka najpierw czy zna już to urządzenie
- pierwszy sterownik urządzenia może rejestrować kolejne dla podurządzeń (urządzenia o wielu funkcjach)

31. Wymień i opisz sposoby komunikacji aplikacji użytkowej ze sterownikiem WDM.

- Buforowany – System operacyjny tworzy bufor niestronnicowany systemu, równy co do wielkości buforowi aplikacji. Do operacji zapisu, I/O Manager User kopiuje dane do bufora systemu przed wywołaniem stosu sterowników. Do odczytu danych, I/O manager kopiuje z bufora systemu do bufora aplikacji po zakończeniu żądanej operacji na stosie sterowników.
- Bezpośredni – System operacyjny blokuje bufor aplikacji w pamięci. Następnie tworzy listę deskryptorów pamięci (MDL), który identyfikuje zablokowane strony pamięci i przekazuje do MDL stos sterownika. Sterowniki mają dostęp do zablokowanych stron przez MDL.
- Własny

32. Omów typowe działanie systemu operacyjnego przy błędach strony (page fault handling).

Błąd strony jest zgłaszany przez sprzęt w czasie, gdy program uzyskuje dostęp do strony, który jest mapowany w wirtualnej przestrzeni adresowej, ale nie załadowany do pamięci fizycznej.

Systemy operacyjne takie jak Windows i UNIX (i inne systemy uniksowe) zapewniają różne mechanizmy raportowania błędów spowodowanych przez błędy strony. Windows w takim generuje wyjątek, natomiast UNIX (i uniksopodobne) zazwyczaj używają sygnałów, takie jak SIGSEGV, do zgłaszania tych błędów w programach.

Jeśli program, który otrzymał błąd nie obsługuje go, to system operacyjny zazwyczaj wykonuje jakieś działanie domyślne (komunikat dla użytkownika oraz rzut pamięci).

33. Na czym polega zagadnienie DLL-hell. Wymień sposoby pozwalające na uniknięcie tego problemu.

Jest to problem, który może się ujawnić przy używaniu dynamicznych bibliotek DLL. Najczęściej jest on spowodowany brakiem lub nieodpowiednią wersją biblioteki (przykładowo biblioteka DLL dostępna jest w trzech wersjach, nazywa się we wszystkich tak samo, ale funkcje przez nią udostępniane się różnią).

Global Assembly Cache jest jednym z rozwiązań. Jest to część platformy CLR. Przykładowo jeśli potrzebujemy dwóch bibliotek o tej samej nazwie (więc nie mogą obie być przechowywane w jednym miejscu na HDD), to można skorzystać z wirtualnego systemu plików należącego do tego rozwiązania. Z poziomu programu będzie to wyglądało tak jakbyśmy korzystali z dwóch różnych bibliotek, ale o tej samej nazwie.

Można też do nazwy biblioteki dodać jej wersję.

34. Wymień trzy podstawowe struktury systemowe wykorzystywane przy budowie i działaniu sterowników urządzeń i podaj ich znaczenie.

DRIVER OBJECT – reprezentuje obraz załadowanego sterownika.

DEVICE OBJECT – reprezentuje urządzenie fizyczne lub logiczne w systemie i opisuje jego charakterystykę

IRP – paczki z danymi, przesyłane między sterownikami.

35. Podaj różnicę pomiędzy zadaniami (jobs), procesami, wątkami i włóknami.

Zadanie jest to nazwany, chroniony i współdzielony obiekt systemu, który pozwala na zarządzanie grupą procesów. Głównym celem zadania jest by grupa procesów mogła być zarządzana jako jedna całość.

Proces jest to wykonujący się program. Jeden proces może należeć do jednego zadania. Jest to kontener z zasobami używanymi podczas wykonywania programu.

Wątek jest to główna jednostka wykonawcza w systemie Windows. Pojedynczy wątek może należeć do jednego procesu, ale proces może zawierać wiele wątków (co najmniej jeden – wątek główny).

Każdy wątek zawiera stan rejestrów procesora, dwa stosy (tryb kernela i chroniony osobno), prywatny obszar przechowywania TLS, unikatowy identyfikator oraz czasami dodatkowy kontekst bezpieczeństwa.

Włókno często bywa nazwane ‘lekkim wątkiem’. Pozwala aplikacji na szeregowanie zadań zamiast zdawać się na globalny dyspozytor. Włókno nie jest widzialne dla jądra, działa jedynie w trybie użytkownika.

36. Jaka jest rola mechanizmu DPC i na czym on polega ?

DPC (Deferred Procedure Call) – odroczone wywołanie procedury; mechanizm, który pozwala zadaniom o wysokim priorytecie (np. obsługi przerwania) na odroczenia wykonania zadania o niższym priorytecie. Pozwala to sterownikom urządzeń i innym niskopoziomowym odbiorcą zdarzeń wykonać część priorytetową ich przetwarzania szybko i dodać do listy zadanie o niższym priorytecie.

Każdy procesor posiada osobne kolejki DPC. DPC ma trzy poziomy priorytetów: niski, średni i wysoki. Domyślnie wszystkie DPC są ustawione na średni priorytet. Wykonanie żądań z kolejki następuje w momencie zakończenia pracy wszystkich przerw o wyższym IRQ.

37. Jaka jest rola mechanizmu APC i na czym on polega?

38. Wyjaśnij następujące pojęcia MBR, VBR, DDK, IDT, GDTR.

MBR – Master Boot Record. Jest to pierwszy sektor na dysku zawierający specjalny program, który jest odczytywany przy starcie komputera do pamięci RAM i którego zadaniem jest przeszukanie tablicy partycji w celu przejścia do VBR.

VBR – Volume Boot Record. Podobny do MBR, ale znajduje się już na konkretnej partycji (na początku) i zawiera program, który powinien rozpocząć ładowanie systemu operacyjnego.

DDK – Driver Development Kit. Zestaw narzędzi pozwalających na tworzenie sterowników urządzeń (i nie tylko) na pewną platformę.

IDT – Interrupt Descriptor Table. Jest to tablica deskryptorów znajdująca się w pamięci RAM, w której zapisane są selektory, które z kolei mają indeksować tablicę GDT, która wskazuje segment, w jakim znajduje się procedura obsługi przerwania.

GDTR – Global Descriptor Table Register. Jest to 48-bitowy rejestr wskazujący położenie w pamięci tablicy GDT (32 bity) oraz limit rozmiaru tej tablicy (16 bitów).

39. Wymień 5 różnych systemów plików.

fat32 - listowy plik do 4GB, dysk do 2TB

reiserfs - journaling, wydajny dla małych plików,

ntfs - hybryda listowej i indeksowej metody (lista kolejnych segmentów pliku zawierających rozmiar ciągłego obszaru zajmowanego przez fragment pliku)

xfs - 64bitowy gwarantowanie pasma dostępu do pliku

ext3 - metoda indeksowa, journaling, można montować jako ext2(narzędzie tune2fs), do 32TB, mniej obciąża procesor niż reiser i xfs

40. Omówić mechanizm przełączania stosu używany w trakcie przyjmowania przerwania.

Mechanizm ten jest identyczny jak w przypadkuwołania funkcji za pomocą furtek. Potrzeba jego użycia zachodzi, gdy program wołany jest bardziej uprzywilejowany niż program wołający. Przebiega następująco:

- nowe wartości rej. SS i ESP przechowywane są w TSS (Task State Segment)
- są one ładowane do SS i ESP, a stare są zachowywane na nowym stosie
- ze starego stosu na nowy kopiowane są parametry podprogramu

- na stosie zapisywany jest ślad (zawartość CS i EIP).

41. Przedstawić klasyfikację maszyn wirtualnych i opisać ich charakterystyki.

Maszyny wirtualne można podzielić na maszyny wirtualne procesowe i systemowe. VM na poziomie procesu dostarcza ABI (Application Binary Interface – rozdziela procesy od systemu operacyjnego) dla aplikacji, a systemowa dostarcza zbioru ISA (Instruction Set Architecture – rozdziela sprzęt od pozostałej części systemu) dla systemu operacyjnego i aplikacji. Procesowe maszyny wirtualne działają jako normalny proces w systemie, są z nim tworzone i usuwane. Ich celem jest przede wszystkim uniezależnienie programowania od platformy, by programy były przenośne mimo różnic sprzętowych (lub też programowych) w maszynach. Przykładem procesowej maszyny wirtualnej może być Java Virtual Machine. Systemowe maszyny wirtualne zezwalają na współdzielenie zasobów sprzętowych między maszyny wirtualne, z których każda może być uruchomiona z innym systemem operacyjnym. Przykładem jest VirtualBox. Wadą tego rozwiązania jest to, że wydajność systemu zainstalowanego na maszynie wirtualnej jest niższa.

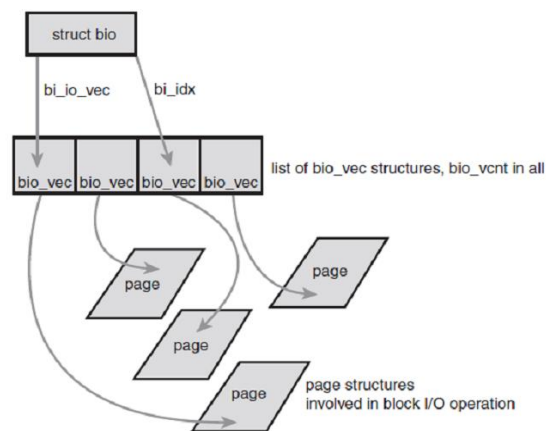


42. Opisz strukturę bio zdefiniowaną w <linux/bio.h>.

Reprezentuje aktywne operacje IO w postaci listy ciągłych obszarów w pamięci (segmentów). Bufory nie muszą zajmować ciągłej przestrzeni w pamięci (scatter-gather I/O). Głównymi elementami struktury są:

- o bio_io_vec - tablica wektorów bio_vec - opisuje ona buffor.
- o bio_vec - wektor opisujący segmenty.
- o Informacje o urządzeniu do którego się odnosi
- o Informacje o typie operacji (read/write)
- o licznik użyc

Struktura nie zawiera informacji o stanie buffora.



43. Porównaj poznane schedulery dla urządzeń blokowych I/O w systemie Linux.

Linus Elevator

W momencie pojawienia się nowego zapytania możliwe są 4 operacje:

- gdy w kolejce znajduje się żądanie do przyległego sektora dysku, żądania te są scalane,
- gdy żądanie znajdujące się w kolejce jest w niej już długo, nowe żądanie jest umieszczane na końcu kolejki – w celu uniknięcia zagłodzenia starszego żądania,
- gdy w kolejce znajdowane są żądania do bliskich sektorów nieprzylegających do siebie ani do sektorów obsługiwanych przez nowe żądanie – jest ono umieszczane pomiędzy tymi żądaniami z kolejki
- gdy nie zachodzą powyższe reguły, żądanie umieszczane jest na końcu kolejki

Deadline I/O scheduler

Każde żądanie jest oznaczone czasem wygaśnięcia (500milisekund dla operacji odczytu, 5 sekund dla operacji zapisu). Poza posortowaną wg kolejności sektorów kolejką dodatkowa implementacja dwóch kolejek FIFO (dla odczytu oraz dla zapisu).

Anticipatory I/O scheduler

Heurystycznie przewiduje operacje, które mogą zaistnieć na podstawie statystyk dla każdego z procesów.

CFQ I/O scheduler

Domyślny scheduler dla kernela 2.6 – osobna kolejka żądań dla każdego procesu, pobieranie kolejnych żądań algorytmem round-robin.

Noop I/O scheduler

Dobry dla urządzeń o losowym dostępie (np. kart pamięci flash) brak sortowania.

44. Czym charakteryzuje się problem „write starving reads” i gdzie występuje?

Problem związany z kolejkowaniem żądań IO przez linux elevator (i nie tylko) (domyślny dla jądra 2.4).

Zapis asynchroniczny, odczyt synchroniczny.

Zapisy w jednej części dysku blokują odczyty w innej (aplikacja czeka na dane).

Deadline zapobiega częściowo zapobiega temu problemowi a anticipatory robi to świetnie.

45. Przedstaw strukturę i-node dla systemu plików ext3.

Każdy i-węzeł zawiera:

- Opis położenia pliku na dysku i rozmiar pliku
- Identyfikator właściciela pliku,
- Typ pliku (pliki zwykłe, katalogi, ...),
- Prawa dostępu do pliku (właściciel pliku, grupowy właściciel pliku, pozostali użytkownicy),
- Czas dostępu do pliku (czas ostatniej modyfikacji, czas ostatniego dostępu, czas ostatniej modyfikacji i-węzła pliku),
- Liczba dowiązań do pliku,
- Liczba procesów, które otworzyły dany plik,
- Inne,
- można zauważyć, że i-węzeł nie zawiera nazwy pliku

46. Podaj blok oraz offset, gdzie znajduje się bajt 6 000 000 w systemie plików ext3. Przedstaw tok obliczania.

bezpośrednie - do 48kb

pośrednie raz - 4mb

pośrednie dwa razy - 4gb

pośrednie trzy razy 4tb

Bajt 6 000 000 wymaga użycia bloku podwójnie pośredniego:

- Pierwszy bajt dostępny jest poprzez blok podwójnie pośredni i ma numer

$$48KB + 4MB = 49\,152 + 4\,194\,304 = 4\,243\,456$$

- Bajt o numerze 6 000 000 jest więc bajtem o numerze 1 756 544 względem początku bloku podwójnie pośredniego

– Każdy blok pojedynczo pośredni daje dostęp do 4MB, więc bajt o numerze 6 000 000 będzie wskazywany pośredni przez pozycję zerową bloku podwójnie pośredniego – po odczytaniu wyznaczonego w ten sposób bloku pośredniego poszukuje się w nim pozycji o indeksie $1\,756\,544 / 4096 = 428$ (reszta 3456),

- Zawartość pozycji o indeksie 428 wskazuje na blok dyskowy zawierający żądany bajt – jest on odległy o 3456 bajtów od początku bloku.

47. Przedstaw proces zamykania procesu w systemie Linux (do_exit()).

do_exit() nam robi z procesu zombie i informuje rodzica o zakończeniu (zostawia mu jakieś dane), jak rodzic je odczyta to wszystko jest zwalniane i proces umiera.

- ustawia flagę PF_EXITING w polu flags w strukturze task_struct,
- del_timer_sync() – deaktywacja timera – w odróżnieniu od del_timer() – gwarantowane, że żaden nie jest zakolejkowany bądź uruchomiony nawet na innym procesorze,
- If BSD process accounting ins enabled, do_exit() calls acct_update_integrals() to write out accounting information,
- exit_mm() – zwolnienie mm_struct, gdy nie jest ona wykorzystywana przez inny proces (współdzielenie) – kernel usuwa ją,
- exit_sem() – gdy proces oczekuje na semafor IPC – jest usuwany z kolejki,
- exit_files(), exit_fs() – dekrementacja licznika użycia obiektów skojarzonych z deskryptorami plików oraz danych dotyczących systemu plików (tak samo jak w przypadku exit_mm – gdy licznik jest 0 – obiekt jest usuwany),

- Ustawienie `exit_code` w `task_struct` na kod przekazany do instrukcji `exit()` bądź kod określający, że proces został przerwany przez kernela. Pole `exit_code` przeznaczone jest do opcjonalnego pobrania przez proces rodzica.
- `exit_notify()` – informowanie procesu rodzica o zakończeniu działania, przypisanie dzieciom nowego rodzica – `forget_original_parent()` – `find_new_reaper()` (procesu znajdującego się w tej samej grupie procesów bądź procesowi `init`), ustawienie `exit_state` w `task_struct` na `EXIT_ZOMBIE`,
- `schedule()` – przełączenie na inny proces – z uwagi, że proces nie jest już poddawany harmonogramowaniu jest to ostatnia funkcja przez niego wykonywana, `do_exit()` nigdy nic nie zwraca.

48. Co to jest proces zombie i jak należy z nim postępować?

Po wykonaniu funkcji `do_exit()` proces przechodzi do stadium `EXIT_ZOMBIE`, pozostaje zaalokowana jeszcze pamięć w postaci: stosu kernela, struktury `thread_info` i `task_struct`. Informacje te po odebraniu przez proces rodzica (`wait()`) bądź po oświadczeniu, że nie jest on nimi zainteresowany pamięć ta jest zwalniana i udostępniana systemowi do ponownego użycia.

W skrócie, aby dobić zombie musimy poprosić o to jego rodzica lub go (rodzica) zabić. Zasoby po procesie zombie zwalnia funkcja `release_task()`;

Znajdowanie procesów zombie `ps - el | grep 'Z'`

49. W jaki sposób w systemie Linux tworzony jest nowy proces?

`fork()` nam robi kopie procesu i ustawia mu PPID na nas (jest naszym dzieckiem), wywołuje `clone()` -> `do_fork()` -> `copy_process()`; `exec()` ładuje kod i zaczyna wykonywanie `spawn = fork + exec`

Bardziej szczegółowy opis:

1. `fork()` – tworzy proces dziecko będący kopią procesu wywołującego tę funkcję (różnica w PID, PPID, niektórych zasobach oraz statystykach, które nie są dziedziczone), implementowany poprzez funkcję systemową `clone()`, która wywołuje funkcję `do_fork()` odpowiedzialną za większość prac związanych z tworzeniem nowego procesu, `do_fork()` wywołuje m.in. funkcję `copy_process()`. Gdy `copy_process()` zakończyła się bezbłędnie – nowy proces jest budzony i uruchamiany.

2. `exec()` – ładuje kod wykonywalny do przestrzeni adresowej i rozpoczyna jego działanie COW - Technika odwlekająca bądź zapobiegająca kopiowaniu danych (każdej strony pamięci w przestrzeni adresowej danego procesu) w momencie tworzenia procesu.

3. przebieg `fork()`:

- `dup_task_struct()` – tworzy nowy stos kernela, `thread_info`, `task_struct` (deskryptor procesu dziecka jest identyczny z deskryptorem procesu rodzica),

- sprawdza czy nowy proces nie wyczerpie zasobów przeznaczonych dla bieżącego użytkownika,

- ustawia niektóre informacje w deskrytorze procesu dziecka na wartości domyślne, np. statystyki.

Większość informacji zawartych w `task_struct` pozostaje jednak niezmienniona,

- stan nowego procesu ustawiany jest na `TASK_UNINTERRUPTIBLE` – zapewnia, że nie jest on jeszcze uruchomiony,

- `copy_flags()` – aktualizacja pola `flags` dla procesu dziecka, `•+ PF_SUPERPRIV = 0, PF_FORNOEXEC=1`,

- `alloc_pid()` – przypisanie nowego PID,

- W zależności od przekazanych flag do funkcji `clone()` – duplikowane bądź współdzielone używane zasoby, informacje dotyczące systemu plików, przestrzeni adresowej, przestrzeni nazw oraz signal handlers,

- zwraca wskaźnik do nowopowstałego procesu dziecka

50. Opisz CFS.

CFS (Completely Fair Scheduler) - scheduler procesów:

- oblicza jak długo proces powinien pracować na podstawie wszystkich procesów

- bierze pod uwagę priorytet

- każdy proces działa przez swoją wagę / suma wszystkich wag

- ustalone targeted latency - czas który dzielimy na wszystkie procesy (mały - lepsza interaktywność ale szamotanie)

- liczy się tylko różnica między priorytetami procesów

51. Zaprezentuj schematycznie i opisz budowę kernela w systemie Linux.

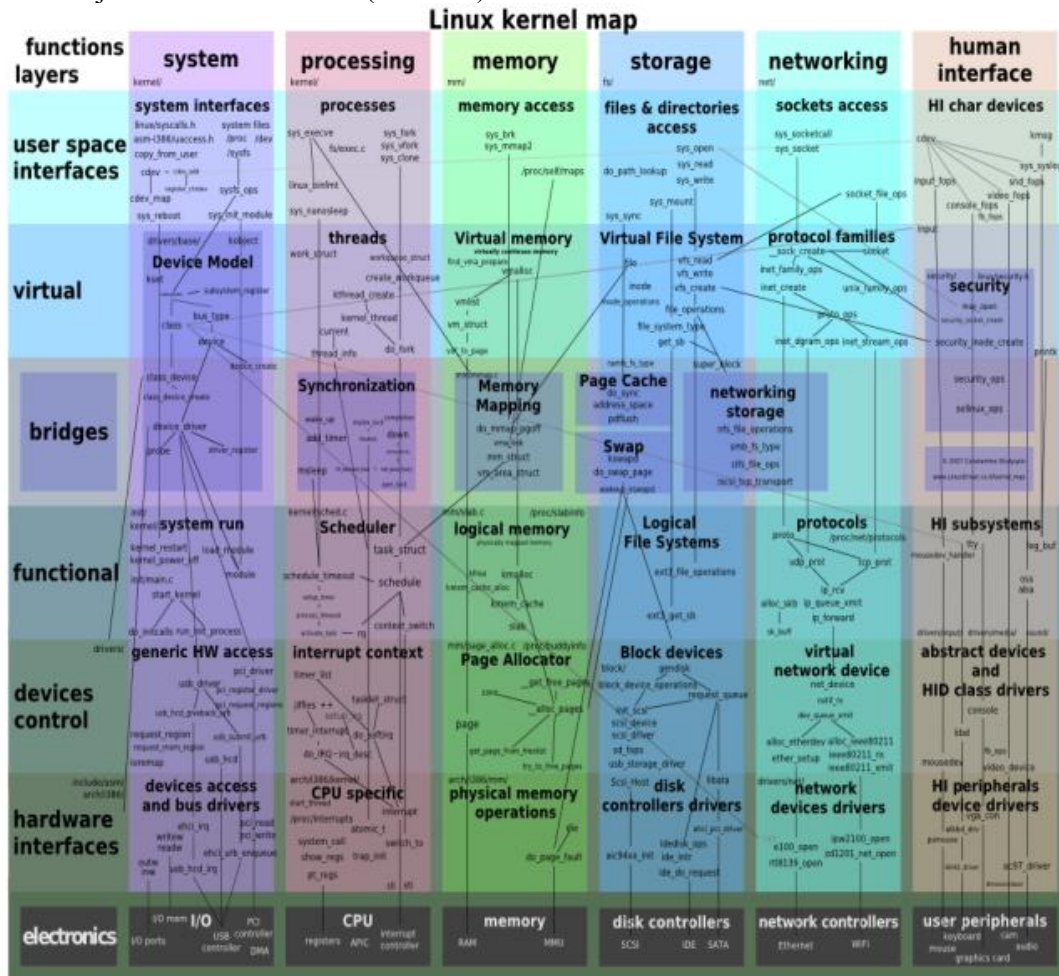
- nie posiada dostępu do biblioteki C ani do standardowych nagłówek języka C, zaimplementowany w GNU C (gcc, Intel C),

- Jądro monolityczne+moduły

- brak ochrony pamięci, (to chyba nie jest do końca prawda)

- posiada trudności z wykonywaniem operacji zmiennoprzecinkowych,

- dla każdego procesu przeznaczona taki sam relatywnie mały stos (x86 – konfigurowalny podczas kompilacji – 4KB lub 8KB – historycznie)
- stos kernela zawiera 2 strony - 8KB na 32 bitowych – 16KB na 64 bitowych architekturach),
- podatny na wyścigi - synchronizacja oraz współbieżność (musi obsługiwać asynchronicznie nadchodzące przerwania, możliwe jest wyłączenie procesu, wspiera SMP),
- powinien zapewniać przenośność na różne architektury,
- ostatnia stabilna wersja Linux kernel 2.6.37 (19.01.11)



52. Podaj przewagę sterowników kernel-space nad sterownikami user-space w systemie Linux.

User-space drivers – wady:

- Niedostępność przerwań (z wyjątkiem niektórych platform)
- Bezpośredni dostęp do pamięci jest możliwy tylko poprzez mmaping (/dev/mem) co może wykonać tylko uprzywilejowany użytkownik.
- Dostęp do portów I/O jest możliwy tylko po wywołaniu ioperm lub iopl. Nie wszystkie platformy wspierają takie wołanie systemowe, a dostęp do /dev/port może być zbyt wolny.
- Wołania systemowe oraz pliku urządzeń dostępne są tylko dla uprzywilejowanych użytkowników
- Czas reakcji jest wydłużony z uwagi na konieczność przełączenia kontekstu przy przesyłaniu danych bądź poleceniu wykonania akcji pomiędzy klientem i urządzeniem.
- Jeżeli urządzenie zostało zrzuczone (swapped) na dysk, czas reakcji jest bardzo długi. Pomoc może użycie wołania systemowego mlock lecz z reguły należy zablokować wiele stron pamięci (programy działające w user-space zależne są od sporego kodu zawartego w bibliotekach).
- Operacja mlock także jest zarezerwowana dla użytkowników uprzywilejowanych.
- Większość najważniejszych urządzeń nie może być obsługiwanych z poziomu użytkownika (włączając w to interfejsy sieciowe oraz urządzenia blokowe)

53. Slab allocator i jego możliwe zachowania.

- Slab allocator dzieli różne obiekty na grupy zwane cache'ami. Każdy z tych cache'ów przechowuje obiekty określonego typu (np. oddzielny cache jest dla task_struct i oddzielny cache dla inode).
- Cache te są podzielone w serie (ang. slabs)

- Slab składa się z pojedynczej strony pamięci.
 - Każdy slab zawiera pewną ilość obiektów, które są zcache'owanymi strukturami danych.
 - Stany slab to pełen (wszystkie obiekty są zaalokowane), połowicznie zajęty bądź wolny (wszystkie obiekty są wolne).
 - Gdy kernel żąda nowego obiektu: połowicznie zajęty, wolny. Gdy nie istnieje wolny slab, tworzony jest nowy
- Slab allocator – cache – możliwe zachowania:
- SLAB_HWCACHE_ALIGN – bezpośrednie przypisanie każdego obiektu w slabie do określonego pola cache – zabezpiecza przed wskazywaniem większej ilości obiektów na tę samą pozycję w cache
 - SLAB_POISON – wypełnienie slaba znaną wartością (a5a5a5a5) w celu wyłapania próby dostępu do niezainicjowanej pamięci (poisoning)
 - SLAB_RED_ZONE – wstawienie stref dookoła zaalokowanej pamięci w cel wykrycia przepełnienia buforu.
 - SLAB_PANIC – w przypadku, gdy alokacja zawiedzie slab allocator panikuje ☹. Używane, gdy alokacja musi zakończyć się sukcesem (np. podczas alokacji struktury VMA w momencie startu systemu).
 - SLAB_CACHE_DMA – nowego slaba należy zaalokować w pamięci, która może być dostępna przy pomocy trybu DMA (czyli gdy nowo alokowany obiekt wykorzystywany jest do DMA i musi znajdować się w strefie ZONE_DMA).

54. Przedstaw szczegółowo oraz porównaj metody alokacji pamięci na poziomie kernela w systemie Linux.

- niskopoziomowe oparte na struct page * alloc_pages(gfp_t gfp_mask, unsigned int order)
 - kmalloc() - wyższy poziom, gdy nie zależy nam na pobraniu konkretnej strony (działanie podobne do malloc() po stronie użytkownika), gwarantuje zwrot stron fizycznie ciągłych
 - vmalloc() - jak kmalloc(), ale zaalokowana pamięć jest wirtualnie ciągła (niekoniecznie fizycznie)
 - slab allocator - cache'owanie często używanych struktur
-
- Gdy potrzebujemy ciągłych fizycznie stron – wykorzystujemy alokatory niskopoziomowe bądź kmalloc()
 - Gdy chcemy alokować na dalekich obszarach pamięci alloc_pages() – kmap() w celu odwzorowania dalekiej pamięci do przestrzeni adresowej kernela
 - Gdy niepotrzebna fizycznie ciągła pamięć – tylko wirtualnie – wykorzystujemy vmalloc()
 - Gdy tworzymy i usuwamy wiele dużych struktur – należy rozważyć slab cache.
 - Slab allocator obsługuje także obiekty cacheowane dla każdego procesora osobno.

55. Przedstaw działanie wybranych flag GFP.

- Modyfikatory akcji:
 - __GFP_WAIT – allocator może przejść w stan uśpienia
 - __GFP_HIGH – allocator może korzystać z awaryjnych pól
 - __GFP_REPEAT – allocator powtarza próbę, jeżeli się nie powiodła
 - __GFP_COLD – allocator powinien wykorzystać zimne strony cache (czyli strony na dysku ciepłe to te w pamięci)
- Modyfikatory strefy (domyślnie alokowane w strefie ZONE_NORMAL):
 - __GFP_DMA – alokuje tylko w strefie ZONE_DMA
 - __GFP_DMA32 – alokuje tylko w strefie ZONE_DMA32
 - __GFP_HIGHMEM – alokuje w strefie ZONE_HIGHMEM bądź w ZONE_NORMAL
- Typy:
 - GFP_ATOMIC – wysoki priorytet, nie może przejść w stan uśpienia.
Wykorzystywana np. przy obsłudze przerw.
 - GFP_NOWAIT – jak w GFP_ATOMIC, tylko nie może wykorzystywać pamięci z zakresu highmem
 - GFP_NOIO – ale nie może inicjować I/O do dysku. Wykorzystywane przy obsłudze dysku, gdy nie można generować więcej żądań do dysku.
 - GFP_NOFS – może inicjować I/O do dysku, ale nie może inicjować operacji na systemie pliku. Wykorzystywane w kodzie obsługi systemu plików, gdy nie można rozpocząć kolejnej operacji na systemie plików.

- GFP_KERNEL – standardowa alokacja. Wykorzystywana w kodzie procesu, gdy bezpiecznie jest uśpić daną operację. (domyślny wybór)
- GFP_USER – alokacja dla procesów użytkownika
- GFP_HIGHUSER – jak w przypadku GFP_USER
- GFP_DMA – alokacja dla strefy ZONE_DMA, gdy urządzenia wymagają pamięci zdolnej do pracy w trybie DMA

56. Co to jest etyka? :)

1. zespół norm i ocen moralnych charakterystycznych dla danej zbiorowości społecznej (np. grupy społecznej, klasy, warstwy, środowiska).
2. filozoficzna nauka o moralności, zajmująca się wyjaśnieniem i ustalaniem takich kategorii, jak dobro i zło, odpowiedzialność, sumienie, powinności oraz wytyczaniem zasad i norm moralnego postępowania.